FINAL PROJECT

TIVIDY: Workflow Management System

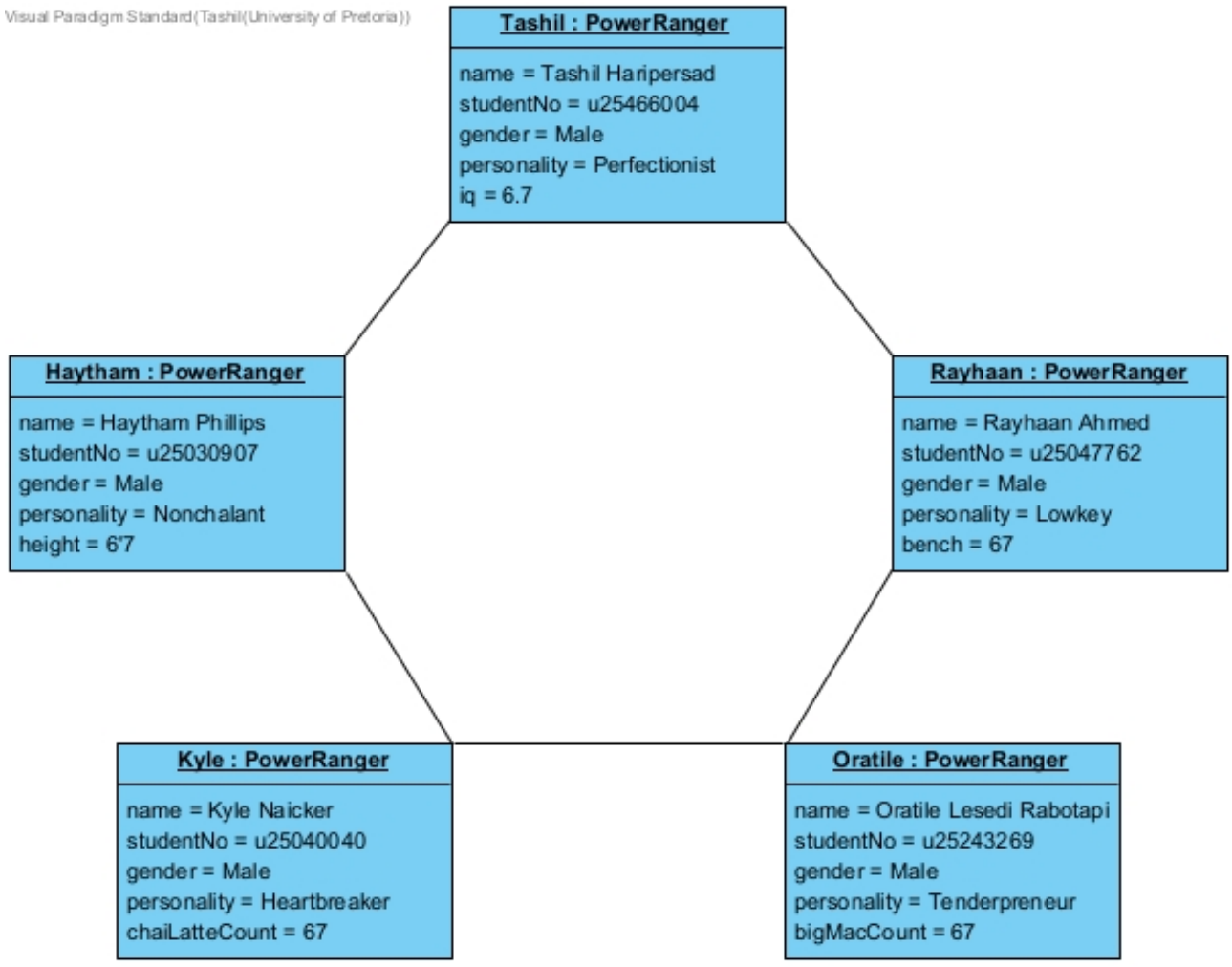


TABLE OF CONTENTS

Research 3
Design Rationale 5
Concrete Scenarios 7
Design Patterns 9
Diagrams 16
Design Decisions and Revisions 26
Testing, Documentation and Github Workflow 32
Team Contribution Statement 33
Build, Run and Test Instructions 34
Bonus Marks 36
References 37

RESEARCH

What is a workflow and what is a workflow management system?

A workflow is a repeatable sequence of tasks carried out in a set order. A workflow management system is the software that runs it, routing each task to the right person and supporting manual and automated work [1], [2]. The WfMC reference model puts a workflow engine at the centre, with interfaces to definition tools, client applications, invoked applications and monitoring tools [2].

What is the difference between a workflow definition and a running workflow instance?

A definition is the model: the activities, their order, the routing rules and the permitted roles. An instance is one running case with its own state, assignees, data and history [2], [3]. Many instances of one definition run side by side without affecting each other [4].

How are activities or work items represented?

An activity is the unit of work in a definition: a task done by a person, a call to another system, or a sub-process containing further activities, all linked by control-flow rules. When the engine reaches an activity, it creates a work item with its own state, assignee and history, which moves from created to completed or failed [5].

How are work items assigned and routed?

Work goes to a role or group, not a named person, and the rule for choosing among eligible participants (such as round robin or shortest queue) is set per task. Routes whose conditions read a participant's response or the instance's data decide the next step, and a parallel split creates child work items that a collector step waits for [6].

How are approvals, rejection and escalation handled?

An approval's outcome, approved or rejected, decides what happens next. The approval path is configurable, either up the organisational hierarchy or as a custom chain of levels, and a rejection sends the work back a level [7].

How are participants, roles and responsibilities represented?

A definition names the role or group allowed to do each activity, and the engine picks a person only when the work is allocated. Each step has one accountable owner with explicit rights to approve, reject or escalate [8].

How are dependencies, events and failures handled?

A step starts only when its dependencies are met: the previous step has finished, and its input data is ready. Failures are handled by safe retries and by resuming from checkpoints, and completed work is undone by a compensation handler instead of a rollback [9].

What makes a workflow engine reusable across different processes?

An engine's core job is the same in every system: interpreting a definition, creating instances and

managing their execution. It can therefore be separated from any one process and embedded in other applications, while everything specific to a process is data [10].

How is workflow history or audit information recorded?

Engines keep a timestamped audit trail of every action, comment and approval for each instance. Reporting only reads this history to measure cycle times and bottlenecks and never changes the running state [11].

DESIGN RATIONALE

Scope of TIVIDY

TIVIDY is a generic workflow engine. A workflow is a tree of work that is built once, copied to give an independent running version, and run to the end. The engine is responsible for:

- **Construction:** building a workflow step by step, and copying a finished one instead of rebuilding it.
- **Structure:** holding the work as a tree of single steps and stages. Top-level steps run one after another and the children of a group run in parallel.
- **Coordination:** deciding in one place what becomes available when something completes.
- **Lifecycle:** keeping every work item to a fixed set of states, so only valid operations are allowed.
- **Assignment:** choosing assignees either by filtering the workflow's participants with strategies, or by taking a list the user supplies.
- **Approval and escalation:** passing a request along a chain of people, one person per command.
- **History and undo:** recording every change as a command, so the newest one can be undone while that is still safe.
- **Reporting:** printing the workflow and summarising its progress without putting report code in the work classes.

Boundaries

TIVIDY only manages the flow of work: what exists, what state it is in, who it is assigned to and what happens next. A few things are deliberately left to the application that uses it.

- **Saving data:** Workflows live in memory. Writing them to a file or database is the application's job.
- **Timers:** Nothing happens by itself when a deadline passes. Escalation is something a person triggers with the Escalate command.
- **Talking to other systems:** If a step involves an outside party, such as Eskom or a referee, it is a work item that a participant completes. TIVIDY never contacts anyone itself.
- **Managing users:** The application creates the participants and owns them. TIVIDY keeps pointers to them and counts their workload.
- **Business logic:** TIVIDY does not know what a repair is or what a good interview looks like. The user supplies ids and names, and roles are plain strings, so each scenario brings its own vocabulary.
- **The user interface:** The GUI sits on top of TIVIDY and uses it through the facade or the classes directly.

Definition vs. instance

A WorkflowDefinition holds the work tree, the participant pool, the mediator and the command history. We have no separate instance class. A finished definition is the template, and cloning it gives an independent running copy.

Belongs to the definition (template)	Belongs to the running copy
The tree of groups and items, with their ids and names	The state of every work item
Which decorators each item carries (audit, escalation, approval)	Assignees, and the workload of participants
The participant pool (shared, not owned)	Escalation and approval chains, and how far each has moved
	Audit logs, the command history and its own mediator

A clone is only allowed while nothing has happened to the template, so every copy starts with all items Created and unassigned. Running one copy never changes the template or another copy.

Expected variation

The shape of each workflow, which decorators each item has, the strategies used for each assignment, the approval and escalation chains, and the reports. The work item lifecycle and the mediator's completion rule are fixed.

Sequential and parallel work

Top-level steps run one after another, and the components inside a WorkGroup run in parallel. Releasing a group makes all its children available at once, and the group is complete only when every child is complete. In WG1 { WI1, WI2 }, WI3, WI4, work items WI1 and WI2 run together, WI3 starts only after both are complete, and WI4 starts after WI3.

CONCRETE SCENARIOS

We use two scenarios. Both are built from the same generic parts. Only the tree, the names, the assignment rules and the chains differ.

Concrete Scenario 1: Incident Response (Power Outage)

- **Environment:** a municipal electricity department responding to a large outage. On 29 September 2026, during heavy rain, an explosion at the Waltloo substation caused a 132 kV trip at Njala and affected at least 11 primary substations [12]. A Njala transformer was found faulty, and the City ran three-hour load rotations while it installed interconnectors to draw supply from other substations [13]. We experienced the outages and the rotation around Hatfield, so we decided to turn our trauma into inspiration and model an incident response workflow for a major municipal power outage, from dispatching the crew to restoring the grid. TIVIDY models the City's response process, not the electrical network.
- **Process being managed:** the response to one outage, from sending out the first team to restored supply.
- **Participants and roles:** dispatcher, engineers, field crew, supervisor and manager. Eskom, emergency services and the equipment supplier are external.
- **Major pieces of work:** five steps, one after another, with no groups.
  1. Dispatch Team: send teams to the affected substations.
  2. Investigation (audited): find the fault and record what is found.
  3. Containment: limit the damage by isolating equipment and starting load rotation.
  4. Repair (audited, escalation, approval): repair or replace the faulty equipment.
  5. Restore: re-energise, end the rotation and tell the public.
- **Decisions and routing points**: whether the crew can repair on site or must escalate, and whether the approvers allow re-energising. Each step ends as passed (completed) or failed (rejected).
- **Approvals and dependencies:** each step starts only when the one before it is complete. Repair can be escalated (supervisor, then manager) and needs approval (supervisor, then manager) before it is complete, so Restore cannot start early.
- **Failures and exceptions:** no suitable engineer is available when work is assigned, the crew cannot fix the fault and the request is escalated, the re-energising attempt fails, as it did on the night [15], and Repair is rejected, an approver rejects the repair (it can be restarted), a step is cancelled, or a step is completed by mistake and the last command is undone while Restore is still Available.
- **External systems:** Eskom, which supplies the City's bulk electricity [14], emergency services, the supplier and public notices. TIVIDY treats each one as a step that a participant performs.
- **Expected outcome:** supply restored, the rotation ended, and a command history and audit logs that act as the audit trail. Otherwise the incident ends Rejected or Cancelled with the same record.

Concrete Scenario 2: Job Application (Tech Company)

- **Environment:** the hiring process of a tech company. We used Tyto Insights, a Pretoria data consultancy that also builds workflow automation [16], as the employer. TIVIDY models only the hiring process.
- **Process being managed:** one candidate's application, from submission to a final decision.
- Participants and roles: recruiter, technical interviewer, behavioural interviewer, hiring manager and director. The candidate and the referees are external.
- **Major pieces of work:** three top-level steps.
  1. Initial Screening (group, parallel): Application Review and Reference Check, both audited.
  2. Interview (group, parallel): Technical Interview and Behavioural Interview, both audited.
  3. Final Decision: needs approval.
- **Decisions and routing points:** each screening or interview step is passed (completed) or failed (rejected), and the final decision is approved or rejected.
- **Approvals and dependencies:** Interview is released only when both screening steps are complete, and Final Decision only when both interviews are complete. The final decision is approved by the hiring manager and then the director.
- **Failures and exceptions:** the candidate fails a step (rejected, so the later stages are never released), the candidate withdraws (cancelled), no interviewer with the right role is available, or an approver rejects the final decision. In the last case the step can be restarted and the approval starts again.
- **External systems:** the candidate, the referees and the channels used to contact them. These are steps that participants perform. TIVIDY does not store documents, interview questions or answers.
- **Expected outcome:** the final decision is approved and the workflow is complete, or the application ends Rejected or Cancelled. The audit logs of the screening and interview steps record what happened.

DESIGN PATTERNS

1. Prototype

Participant	Class
Prototype	WorkflowPrototype
ConcretePrototype	WorkflowDefinition
Client	TividyFacade

- **Problem:** A finished workflow, with its tree, decorators, participants and mediator, takes a lot of calls to assemble. Every outage or application also needs its own copy, so that running one never changes the template or another run.
- **Collaboration:** The client asks a WorkflowDefinition to clone itself. The clone deep-copies every step (groups copy their children, decorators copy what they wrap), shares the participant pointers, and gets its own mediator and an empty command history. Every work item starts Created with no assignees or chains. Cloning is refused with a NotPristineException once any item has moved.
- **Varies:** The content of each workflow, and how many runs are made from it.
- **Why:** Rebuilding the Power Outage workflow for every outage would repeat the same builder calls and risk small differences between runs. A clone gives the same structure with its own state.

Note: Some classes have a copy() method. It is only deep-copy support for this pattern and does not make them Prototype participants.

2. Builder

Participant	Class
Builder	Builder
ConcreteBuilder	WorkflowBuilder
Director	WorkflowDirector
Product	WorkflowDefinition

- **Problem:** Workflows contain groups inside groups and items wrapped in up to three decorators, and users must be able to build their own by supplying ids and names. Writing that nesting by hand is easy to get wrong.
- **Collaboration:** The Director (or user code) calls reset, then beginGroup, newWorkItem, the with… calls, endWorkItem and endGroup. The builder keeps a stack of open groups, rejects repeated ids (DuplicateIdException) and always wraps decorators in the same order, Audited, Escalated, Approved, WorkItem, whatever order the calls came in. getResult checks that nothing is left open and hands the product to the caller.
- **Varies:** The shape of the tree and the decorators on each item.
- **Why:** The construction rules (unique ids, no empty groups, wrapping order) live in one place, so both scenarios and any custom workflow are built the same way.

Note: getResult sits on WorkflowBuilder and not on the Builder interface, because each concrete builder returns its own kind of product. For the same reason the Director's methods return nothing.

3. Composite

Participant	Class
Component	WorkComponent
Composite	WorkItem
Director	WorkGroup
Client	WorkMediatir, Command, WorkViistor, WorkflowDefinition

- **Problem:** A workflow is made of single steps and stages that contain more steps, some running in parallel and the rest in sequence. The mediator, the commands and the reports should not care whether they are holding Technical Interview or the whole Interview stage.
- **Collaboration:** A WorkGroup owns its children and passes release on to all of them. It is complete when every child is complete, and groups can nest. WorkflowDefinition keeps the top-level steps in order.
- **Varies:** The nesting depth and the size of each group.
- **Why:** The Job Application workflow mixes single steps with parallel stages. One interface lets the rest of the system treat both the same way, with no type checks.

Note: WorkItemDecorator is also a WorkComponent (see Decorator). Item-only operations on a group throw UnsupportedOperationException, which is the usual price of giving every component one interface.

4. Iterator

Participant	Class
Iterator	WorkIterator
ConcreteIterator	DepthFirstIterator
Aggregate	WorkComponent
ConcreteAggregate	WorkItem, WorkGroup, WorkItemDecorator, WorkflowDefinition

- **Problem:** Anything that needs the whole tree, such as listing the steps or counting items, should not have to know how a group stores its children or how decorators wrap items.
- **Collaboration:** createIterator returns a DepthFirstIterator. It keeps a stack of components still to visit and walks a group, then its children, then the next sibling. A decorated item appears as its outermost decorator. The caller deletes the iterator.
- **Varies:** The traversal order.
- **Why:** If the way children are stored changes, only the iterator changes.

Note: There is no separate Aggregate class. WorkComponent is already the common interface for everything we iterate over, and WorkflowDefinition declares createIterator itself, so an extra base class with one method added nothing.

5. Visitor

Participant	Class
Visitor	WorkVisitor
ConcreteVisitor	ProgressReport, TreePrinter
Element	WorkComponent

ConcreteElement	WorkItem, WorkGroup, AuditedWork, EscalatedWork, ApprovedWork
ObjectStructure	WorkflowDefinition

- **Problem:** A manager wants a progress summary and a readable picture of the workflow, showing what is complete and which steps are audited or need approval. Putting that code in the work classes would bloat them, and each new report would change them.
- **Collaboration:** WorkflowDefinition::accept passes the visitor to every step. A group visits itself and then its children. A decorator announces itself with its own visit overload and then forwards to the item it wraps. ProgressReport counts items per state. TreePrinter collects the decorator tags and prints one indented line per item and group.
- **Varies:** The set of reports.
- **Why:** A new report is a new visitor class, and the work classes stay the same.

Note: The Element is WorkComponent, which is also the Component of the Composite. A separate Element class would only copy the interface every client already uses.

6. State

Participant	Class
Context	WorkItem
State	WorkItemState
ConcreteState	CreatedState, AvailableState, AssignedState, InProgressState, EscalatedState, ApprovalState, CompletedState, RejectedState, CancelledState

- **Problem:** A work item can be created, available, assigned, in progress, escalated, in approval, completed, rejected or cancelled, and what may be done to it depends on that. Completing an item nobody started, or starting a completed one, must be refused.
- **Collaboration:** A WorkItem passes each request to its current state object. A legal request calls setState with the next state, and anything else throws InvalidTransitionException. setState also keeps participants' workloads correct when an item enters or leaves an end state.
- **Varies:** The states and the transitions between them.
- **Why:** Without State, every operation on a work item would begin with a chain of checks on a status field.

7. Strategy

Participant	Class
Context	WorkItemState
Strategy	AssignmentStrategy
ConcreteStrategy	RoleAssignment, LeastLoadedAssignment, RuleAssignment

- **Problem:** Who should do a step depends on the organisation. In Incident Response, Investigation should go to engineers with the lightest workload, while an interview in

Job Application needs someone with the right role. Hard-coding one rule would mean rewriting the work item whenever the rule changes.

- **Collaboration:** Strategy assignment always works on the workflow's whole participant pool. The Assign command passes the pool, the ordered strategies and the number of assignees. The item copies the pool and applies each strategy in order, each narrowing the previous result. RoleAssignment keeps holders of a role, LeastLoadedAssignment sorts by workload, and RuleAssignment replaces the final pick with a random one. Then the first numAssignees are taken (all of them if -1). Asking for more people than remain throws InsufficientParticipantsException, and bad counts, duplicates or null entries throw InvalidAssignmentArgumentException. If no strategies are given, the user passes the participants explicitly instead.
- **Varies:** The strategies and their order.
- **Why:** "Role, then least loaded" is two small classes in a list and not a new class for every combination.

Note: The Context does not store its strategies. The Assign command hands them to assignWork each time, and the caller owns them. The right rule can differ for every assignment, and an assignment can be undone and done again.

8. Chain of Responsibility

Participant	Class
Handler	Handler
ConcreteHandler	LevelHandler, FallbackHandler
Client	EscalatedWork, ApprovedWork

- **Problem:** A repair the crew cannot finish has to be passed up to someone who can, and a repair needs sign-off from several levels. Who is asked next should be data, not code.
- **Collaboration:** When work is assigned, the user supplies one participant list for the escalation chain and one for the approval chain. They are independent of the assignees and are never filtered by strategies. A LevelHandler is made for each participant, and an escalation chain ends with a FallbackHandler. The first Escalate command moves the item to Escalated and gives the request to the first handler. Each later Escalate means the previous handler could not resolve it, so the next one takes over. When the next handler is the FallbackHandler, the item completes immediately, and a Complete command also stops the escalation. The first Approve command moves the item to Approval and gives it to the first approver, each later one means the previous approver signed off, and reaching the last approver completes the item. Approval chains have no fallback, and an empty one is refused.
- **Varies:** The participants, their order and the number of levels.
- **Why:** One, two or three levels need no code change, and the same mechanism handles both escalation and approval.

Note: The textbook walks the whole chain in one call. Ours moves one link per Escalate or Approve command, because each link is a person acting at a different time. The decorator keeps the position and the chain.

9. Decorator

Participant	Class
Component	WorkComponent
ConcreteComponent	WorkItem
Decorator	WorkItemDecorator
ConcreteDecorator	AuditedWork, EscalatedWork, ApprovedWork

- **Problem:** Some steps need extra behaviour and others do not. Repair needs auditing, escalation and approval, Dispatch Team needs none of them, and Application Review needs only auditing. A subclass for every combination would mean eight classes for three behaviours, and more each time we add one.
- **Collaboration:** A decorator owns the component it wraps and forwards every operation, adding its own step. AuditedWork logs each successful operation, and users can add, edit and delete log entries. EscalatedWork and ApprovedWork own their chains. The builder always wraps Audited around Escalated around Approved around WorkItem, so that EscalatedWork can reach ApprovedWork. When an escalation ends on an item that also has approval, EscalatedWork calls approve, so the flow is Escalate, Approve, Completed. find and getItems return the outermost decorator, so commands always pass through every layer.
- **Varies:** Which behaviours each item carries.
- **Why:** Items get exactly the behaviour they need, and the other classes do not change.

Note: A WorkGroup is also a WorkComponent and could be wrapped, but our scenarios only decorate work items.

10. Mediatior

Participant	Class
Mediator	Mediator
ConcreteMediator	WorkflowMediator
Colleague	WorkComponent

- **Problem:** Completing one step must make the next one available, and with parallel groups the next step may only open once every sibling is done. If items knew their siblings and successors, the steps would be tangled together.
- **Collaboration:** A completed WorkItem only notifies the mediator. The mediator finds the item's top-level step and checks whether it is complete (a group is complete when all its children are). If it is, the mediator releases the next step, and if not it does nothing. In WG1 { WI1, WI2 }, WI3, WI4: WI2 completes and nothing happens, WI1 completes so WG1 is complete and WI3 is released, and WI3 completing releases WI4. Rejected and cancelled items never complete, so nothing after them is released.
- **Varies:** The rule that decides what becomes available.
- **Why:** The completion rule is in one class, and work items know nothing about their neighbours.

11. Command

Participant	Class
Command	Command, WorkCommand

ConcreteCommand	StartCommand, CompleteCommand, RejectCommand, CancelCommand, EscalateCommand, ApproveCommand
Client	TividyFacade
Invoker	CommandHistory
Receiver	WorkComponent

- **Problem:** Every assignment, start, completion and rejection has to be recorded, and the most recent change has to be undoable when a step is completed by mistake.
- **Collaboration:** The client creates a command with its target, and the workflow passes it to CommandHistory, which runs and stores it. undoLast undoes the newest command and removes it, so undo can be repeated. A refused undo leaves the command on the stack. Commands target work items (decorated ones included), and a group target throws UnsupportedOperationException. AssignCommand has two forms: one that uses the workflow's pool and strategies, and one that takes an explicit participant list. Commands print what they do to the console.
- **Varies:** The operations, and what is logged.
- **Why:** The list of descriptions is the audit trail, and an operation that is an object can be listed and undone.

12. Memento

Participant	Class
Originator	WorkItem, EscalatedWork, ApprovedWork
Memento	WorkMemento
Caretaker	WorkCommand

- **Problem:** Undo has to put back the item's state, its assignees, the participants' workload and the chains of its decorators, without exposing the item's internals.
- **Collaboration:** Before a command performs, it asks each affected component for a memento. The request passes through the decorators and each layer adds its own data. undo hands the mementos back in reverse order. The affected set is the target, plus the items of the next step for Complete, Escalate and Approve, since those can release work. A command can be undone only if every affected component is still in the state the command left it, so a completion can be undone only while the work it released is still Available. A command that fails halfway is rolled back with the same mementos.
- **Varies:** What each command captures.
- **Why:** Only the originators can read a snapshot, so the caretaker stays ignorant of it, and the states need no special revert operation.

13. Template Method

Participant	Class
AbstractClass	WorkCommand, Handler
ConcreteClass	StartCommand, CompleteCommand, RejectCommand, CancelCommand, EscalateCommand, ApproveCommand, LevelHander, FallbackHandler

- **Problem:** Every command must follow the same safe sequence (print, snapshot, act, record the result, roll back on failure), and every chain must be walked the same way. If each command wrote its own version, one of them could forget the snapshot and undo would quietly break.
- **Collaboration:** execute collects the affected components, takes mementos, calls perform and records the resulting states. undo checks canUndo before it restores anything. handle asks canHandle, runs process if it can, and otherwise passes the request on.
- **Varies:** perform, canUndo, getAffected, canHandle, process and cloneSelf.
- **Why:** The snapshot and the undo check cannot be skipped by a new command.

Note: perform, canHandle, process and cloneSelf are abstract. canUndo and getAffected are hooks with default behaviour, and the Complete, Escalate and Approve commands override getAffected.

14. Façade

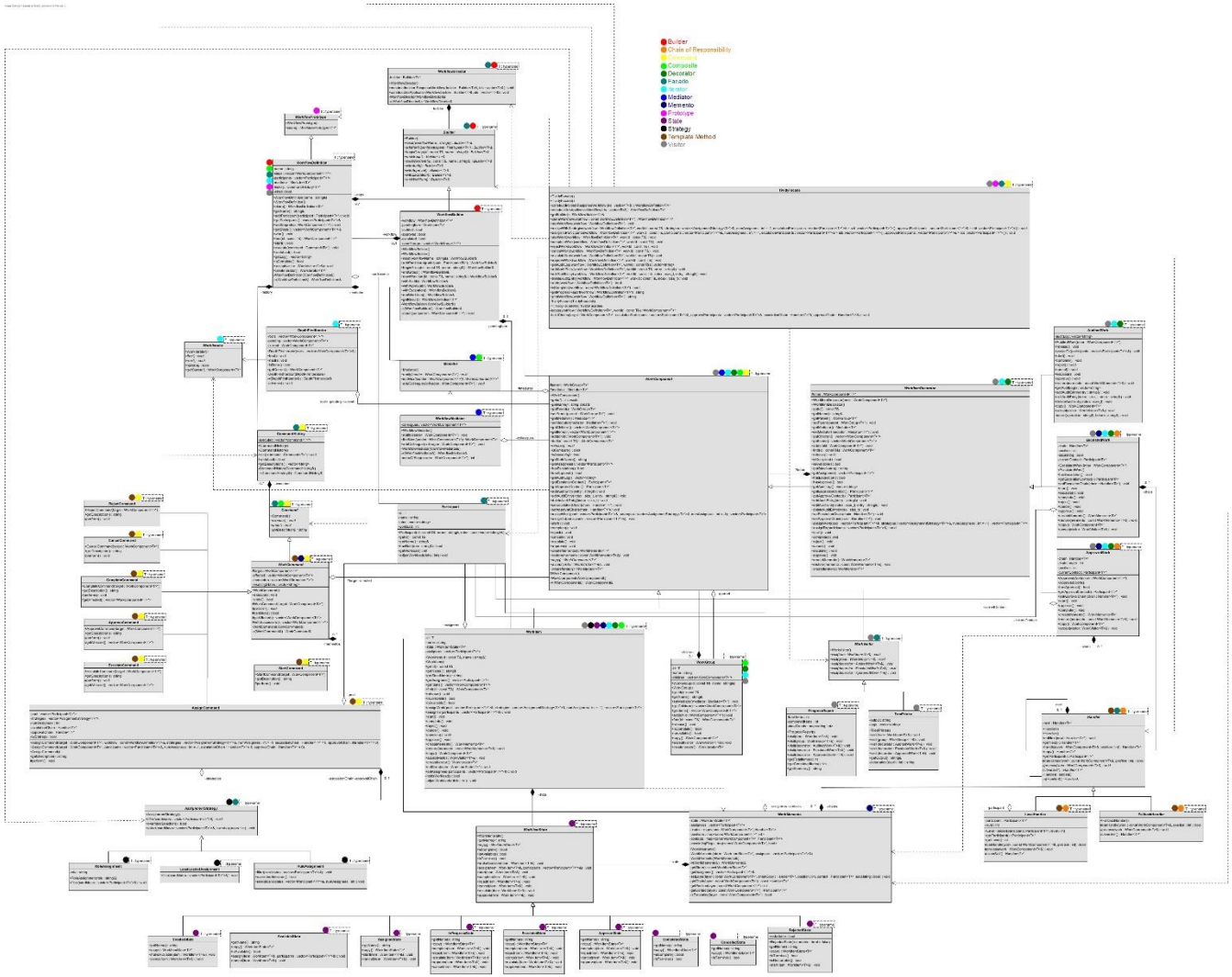
Participant	Class
Façade	TividyFacade
Subsystem classes	Builder, WorkflowDirector, WorkflowDefinition, Participant, AssignmentStrategy, Handler, Command, CommandHistory, WorkVisitor

- **Problem:** Without a single entry point the GUI would need to know the builder, the director, the commands, the handler chains and the visitors just to run one workflow.
- **Collaboration:** The facade owns one Builder and builds both scenarios through the Director. getBuilder gives the GUI the builder for custom workflows. assignWithStrategies and assignToParticipants turn the escalation and approval participant lists into handler chains (adding the FallbackHandler to escalation chains) and run an AssignCommand. startWork, completeWork, rejectWork, cancelWork, escalateWork and approveWork look the item up by id (ComponentNotFoundException if it does not exist) and run the matching command. The facade also exposes the audit log methods, undo, the progress report and the printed tree. It is a convenience, and the same calls can be made on the subsystem classes directly.
- **Varies:** The internals behind the interface.
- **Why:** The GUI depends on one class and not on the dozens behind it.

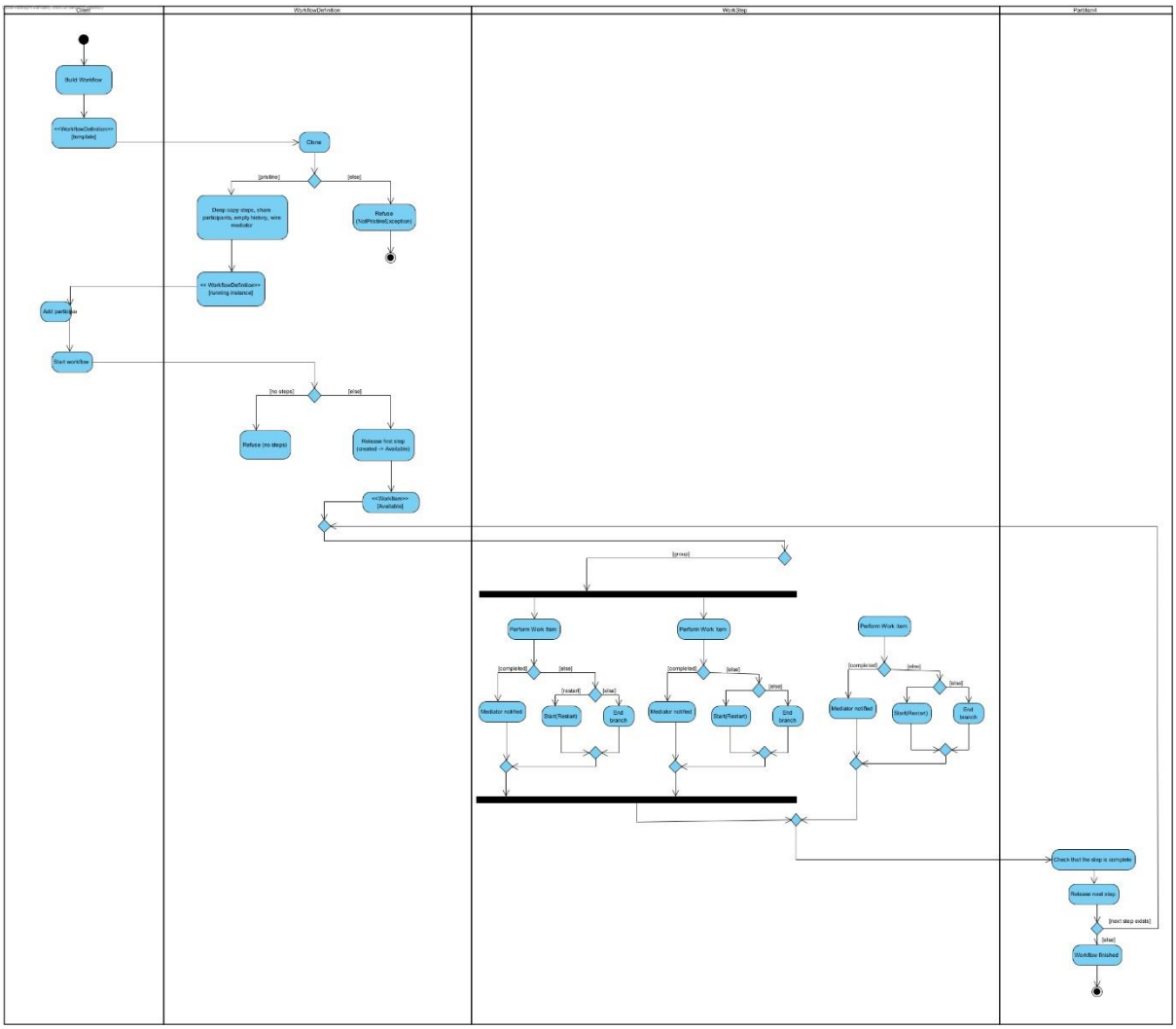
These are not patterns. Participants are created and owned by the application, and each one gets its id, name and roles in the constructor. Workflows only keep pointers to them. All errors derive from TividyException, with one subclass for each kind of problem, so the GUI can catch one type and show the message.

DIAGRAMS

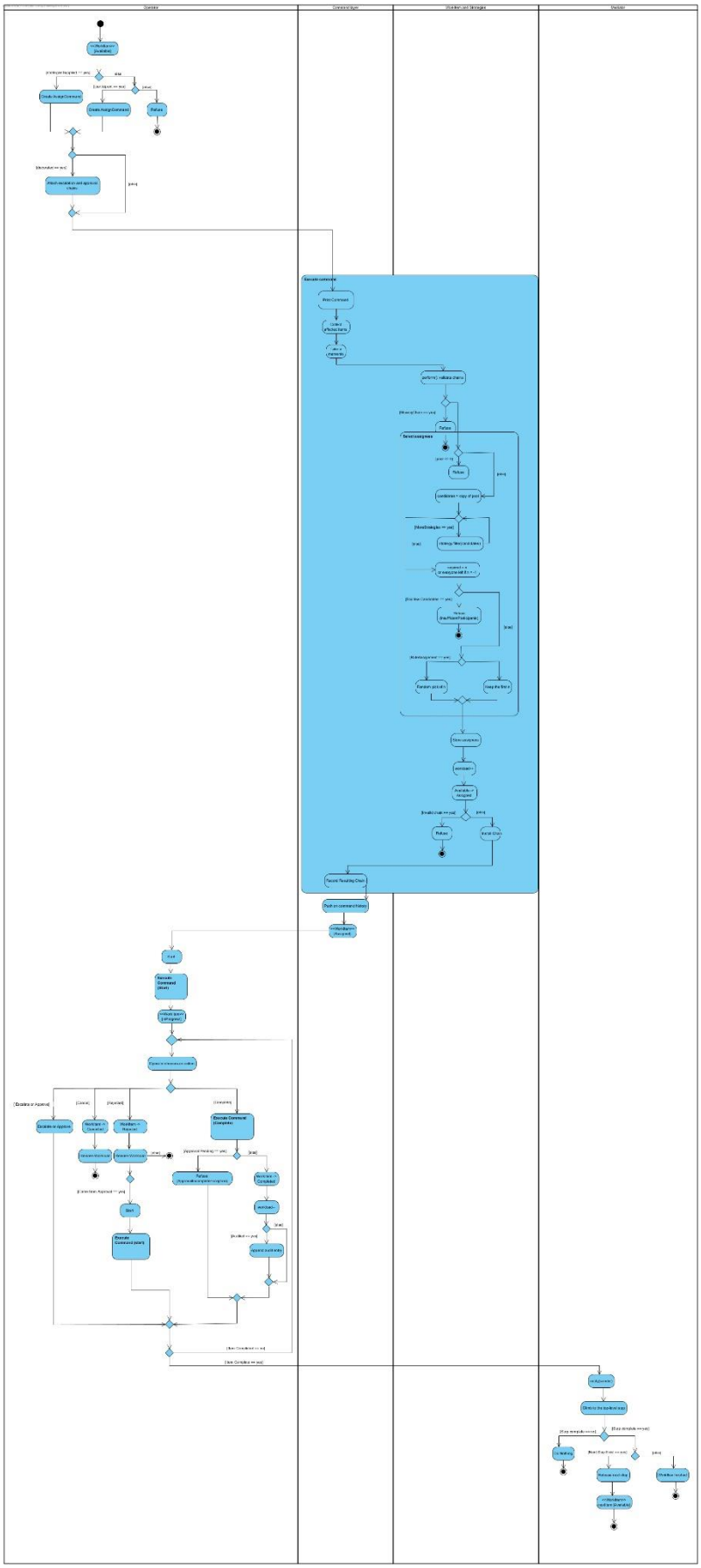
Class Diagram



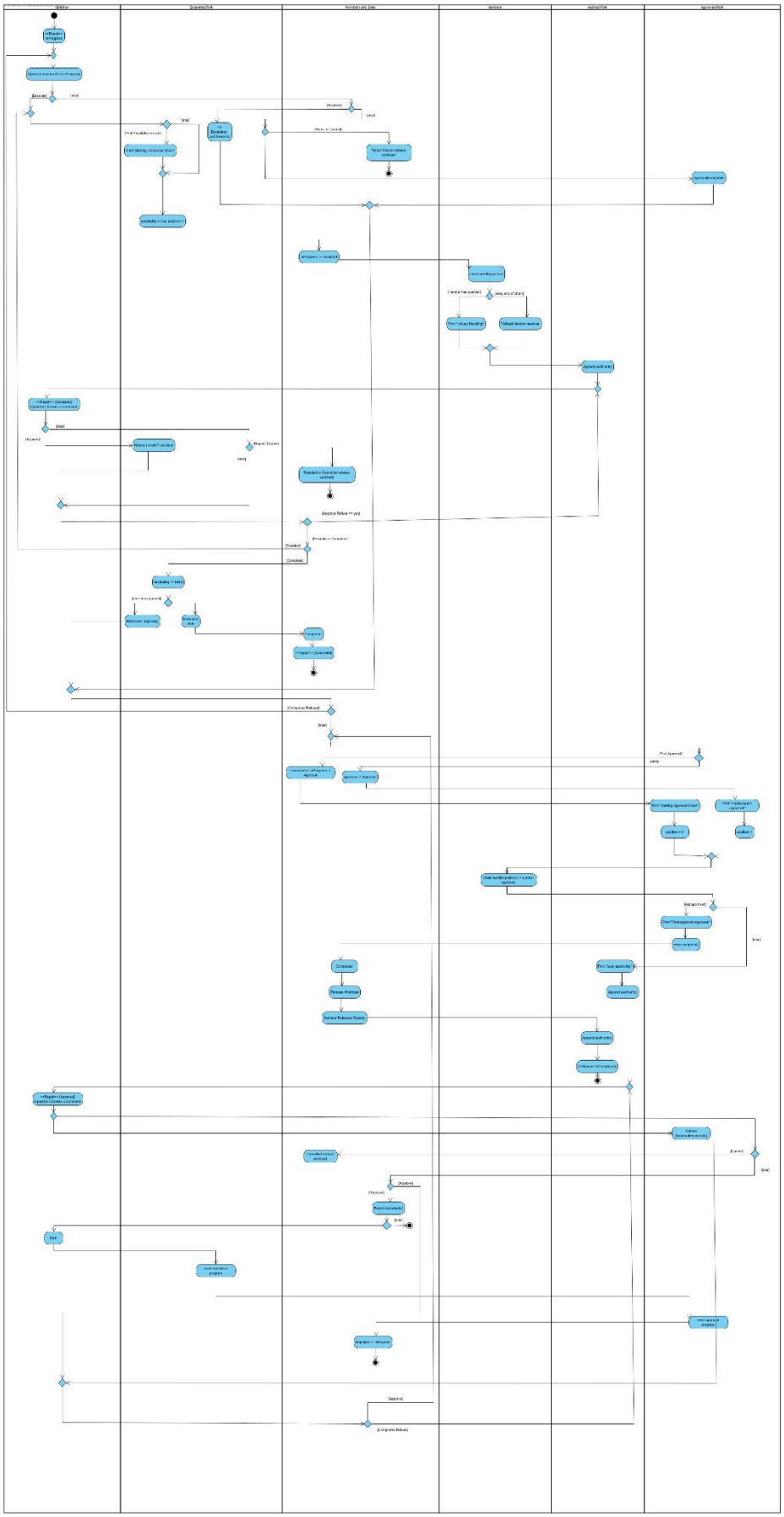
Activity Diagram 1



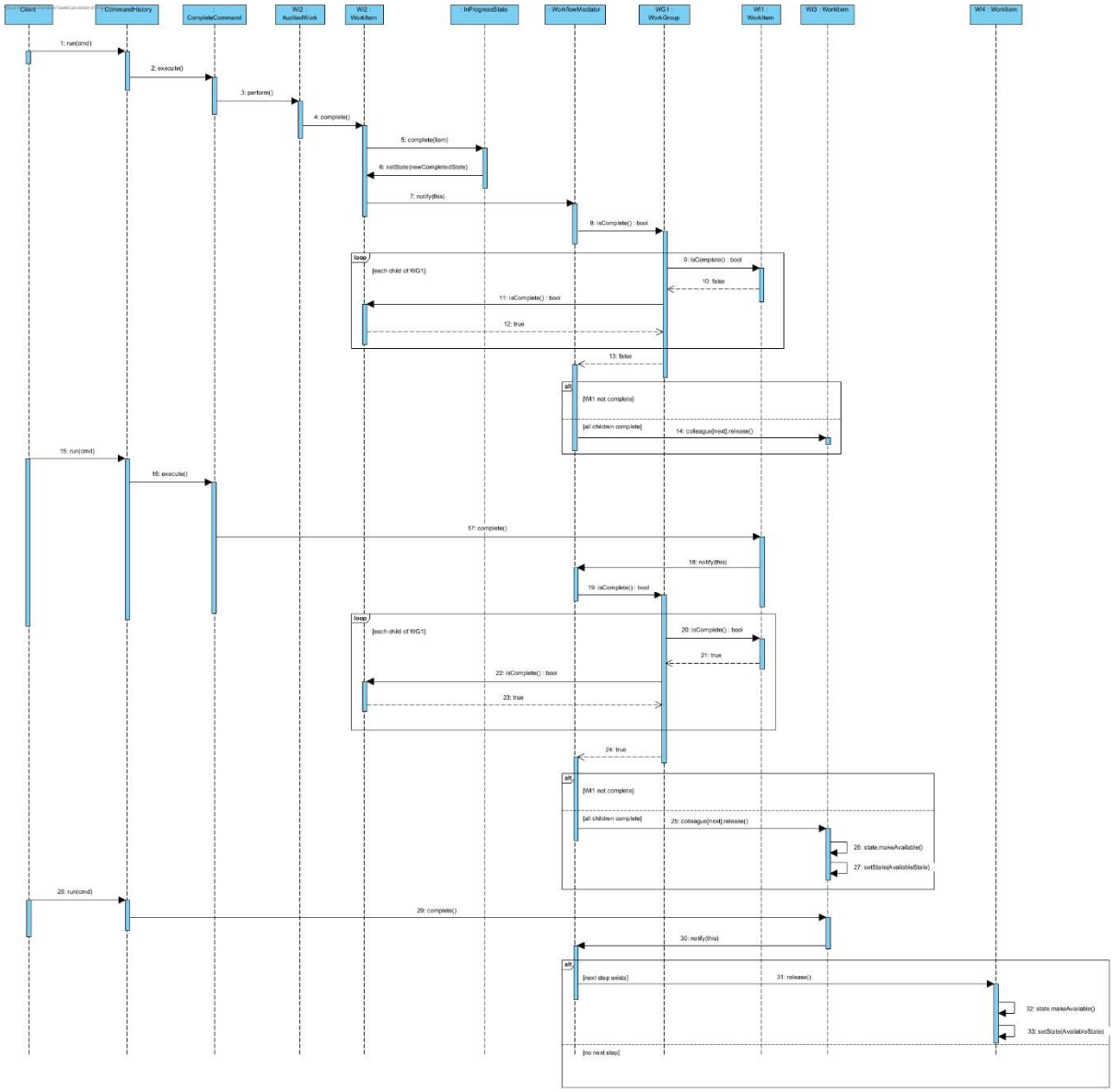
Activity Diagram 12



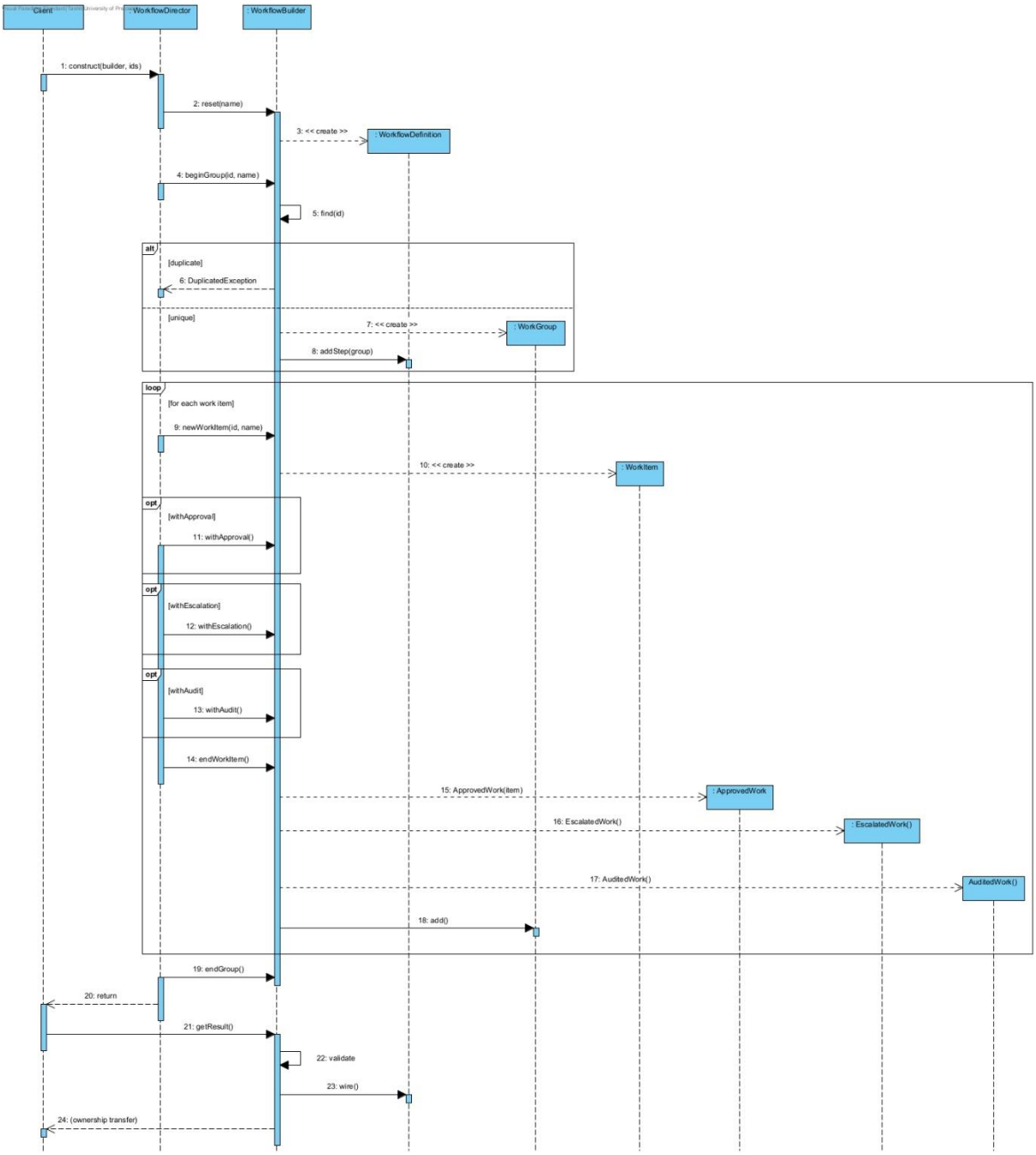
Activity Diagram 3



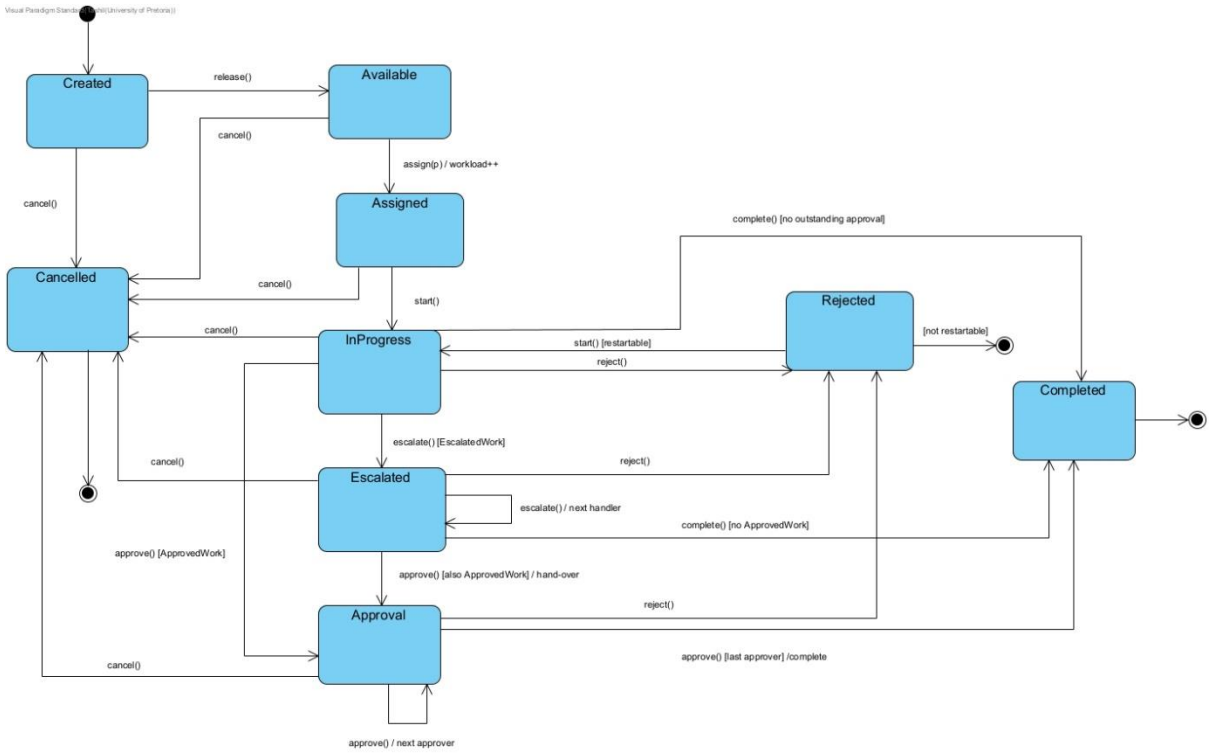
Sequence Diagram 1



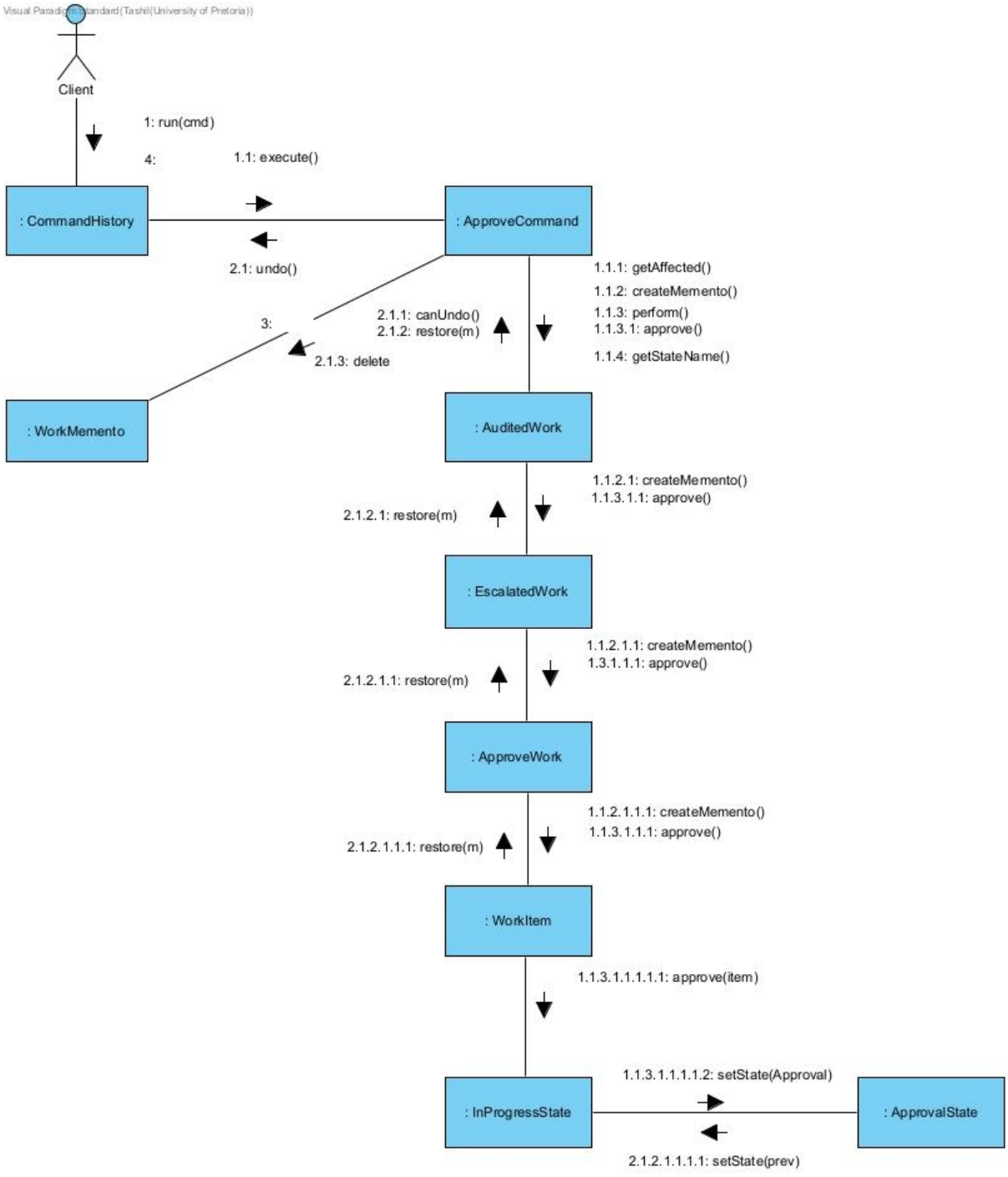
Sequence Diagram 2



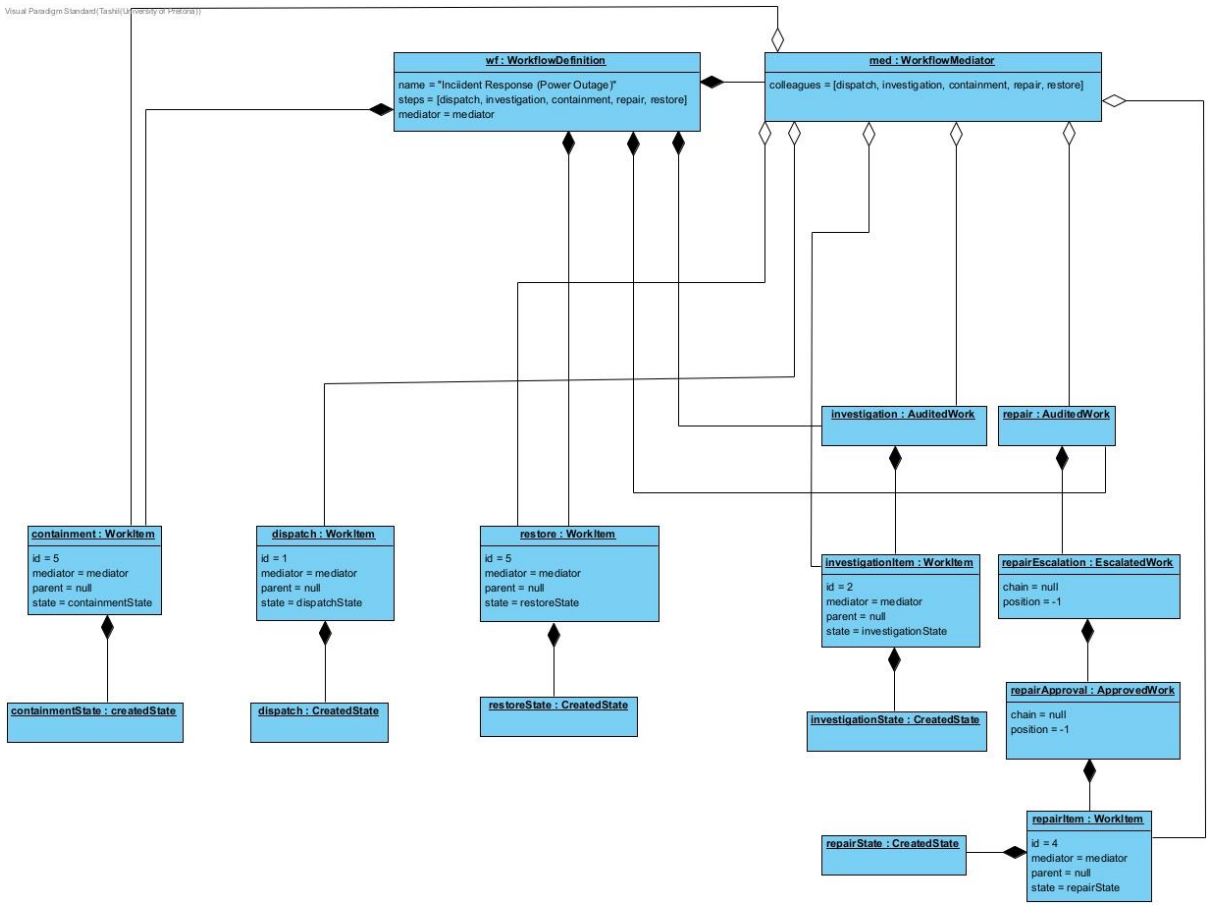
State Machine Diagram



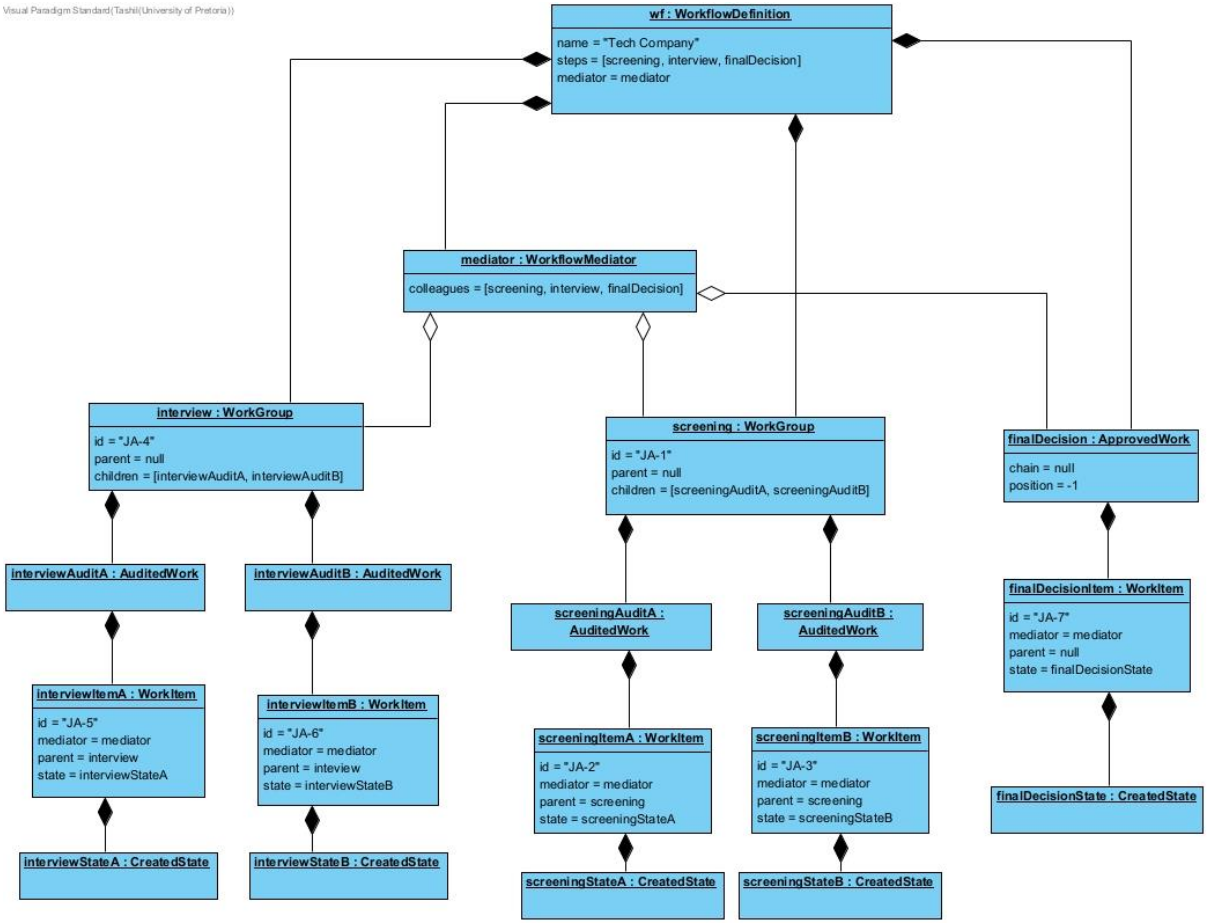
Communication Diagram



Object Diagram 1



Object Diagram 2



DESIGN DECISIONS AND REVISIONS

Initial design decisions

- **Lifecycle operations live on WorkComponent:** Question: how can Decorator wrap tasks? Decision: start, complete and similar operations are on the common interface, not only on WorkItem, so a decorator is interchangeable with a task. Groups implement them by delegating to children. Diagrams: class diagram.
- **Definitions are immutable and versioned:** Decision: running instances keep the version they started with, as in [3]. Diagrams: class diagram, sequence diagram for creating an instance.
- **Undo covers leaf work items only:** Groups derive their status from their children, so a group cannot be snapshotted. Diagrams: class diagram.
- **Roles and event types are strings:** This keeps the engine generic and lets scenarios define vocabulary. Trade-off: no compile-time checking.

Revisions

- **Scope of the model: Problem:** should TIVIDY model the electrical network? **Original:** unclear. **Change:** it models the response process only, and network details travel as data. **Why:** the brief says the scenario must not determine the architecture.
- **Scenario choice.** (superseded by R22 and R45) **Problem:** which scenario is realistic and relatable? **Original:** a placeholder (an insurance claim). **Change:** substation fault response in Tshwane. **Why:** we lived it, and it needs parallel work, approval, escalation and undo. **Diagrams:** activity diagram 3.
- **R2. Who runs the process. Problem:** we assumed the national department handled outages. **Original:** that department. **Change:** the City's electricity unit, with Eskom as an external party, after research showed the department deals with policy while Eskom and the municipalities run the grid. **Why:** accuracy.
- **R3. Scope of the model. Problem:** should TIVIDY model the electrical network? **Original:** unclear. **Change:** it models the response process only. **Why:** the scenario must not decide the architecture.
- **R4. Terminology. Problem:** Eskom's “load reduction” is not Tshwane's “load rotation” [9]. **Original:** we used the terms loosely. **Change:** we say load rotation. **Why:** it matches our source.
- **R5. Raw pointers. Problem:** ownership must be explicit. **Original:** unique_ptr, shared_ptr and std::function. **Change:** raw pointers only, with destructors, deleted copies on owning classes and written ownership rules. **Why:** clear lifetimes. **Diagrams:** class diagram (composition and aggregation).
- **R6. Observer replaced by Mediator. Problem:** how does completing one step release the next? **Original:** items published events on an event bus. **Change:** a completed WorkItem notifies WorkflowMediator, which checks the group and releases the next step. **Why:** the group-completion rule belongs in one place. **Diagrams:** class, activity and sequence diagrams.

- R7. Adapter, connectors and events removed. Problem: how to reach external systems. Original: Connector, LegacyApiAdapter, EventBus and Event. Change: all removed, and external parties are steps that participants perform. Why: an adapter needs a known external interface, which a generic system cannot assume. Diagrams: class diagram, activity diagram 3.
- R8. Scope reduced to a generic workflow. Problem: the design modelled a whole system. Original: TividyEngine and WorkflowInstance owned definitions, instances and connectors. Change: both removed, and a running workflow is a clone of a WorkflowDefinition. Why: the brief asks for a generic workflow. Diagrams: class diagram, activity diagram 1.
- R9. Prototype moved. Original: work components cloned themselves. Change: WorkflowDefinition::clone copies the whole workflow. Why: a workflow is much more work to build than one item. Diagrams: class diagram.
- R10. Strategy ownership. (superseded by R33) Original: one strategy per definition. Change: each WorkItem held an ordered vector of strategies. Why: different tasks need different rules.
- R11. Decorators and chains. (chain participants changed in R35) Original: ValidatedWork, AuditedWork and NotifyingWork. Change: AuditedWork, ApprovedWork and EscalatedWork, the last two owning a handler chain. Why: approval and escalation belong with the behaviour they add, and notification depended on the removed events. Diagrams: class diagram.
- R12. Lifecycle and undo. Original: eight states, and undo restored only the command's target. Change: an Approval state was added, and the last command could be undone whatever it was. A completion is undoable only if the next component is still Available. Why: undoing a completion must not leave later work already started. Diagrams: state and class diagrams.
- R13. Template Method added. Problem: after R6 and R7 we were short of patterns. Change: WorkCommand::execute and undo, and Handler::handle, became template methods. Why: every command snapshots and checks canUndo the same way. Diagrams: class diagram.
- R14. Builder and Facade roles. (superseded by R30) Original: scenario code directed the builder. Change: TividyFacade became the Director.
- R15. Sequential and parallel work. Original: WorkGroup had a Sequential or Parallel mode. Change: children of a group run in parallel and everything else runs in sequence. Why: one fixed rule needs no enum and matches the mediator example. Diagrams: class diagram, activity diagram 1.
- R16. Templates and naming. Original: ids were strings and enums existed. Change: every class is a template on the id type, there are no enums, and attribute names have no underscores. Why: an id may be an int or a string. Diagrams: class diagram.
- R17. Approve command added. (rules refined in R36) Problem: approval happened inside Complete and could not be recorded. Change: an ApproveCommand was added, and Complete on an approved item is refused until the approval ends. Why:

approval is a user action that should be recorded and undone. **Diagrams:** class, state and activity diagrams.

- **R18. Assignees are a list and chains are built at run time.** (chain part superseded by R35) **Original:** one assignee per item and ready-made chains. **Change:** a list of assignees, and decorators that start without a chain. **Why:** who works and who approves is known only when the user acts. **Diagrams:** class and activity diagrams.
- **R19. Moving along a chain.** (refined in R36) **Original:** Escalated and Approval were left by a single assign or complete. **Change:** each accepts its own request again to move to the next handler. **Why:** repeated commands must be able to walk the chain. **Diagrams:** state diagram.
- **R20. Undo of chain commands.** (extended in R28 and R39) **Original:** a memento held the state and assignee. **Change:** it also records each chain decorator's position. **Why:** Escalate and Approve change the decorators. **Diagrams:** class and communication diagrams.
- **R21. Template parameter name. Original:** IdType. **Change:** T. **Why:** team convention.
- **R22. Scenario made general.** (superseded by R45) **Problem:** our mentor said the scenario should be general. **Change:** a general Incident Response System, with the power outage only in the demo's console messages. **Diagrams:** activity diagram 3.
- **R23. Number of assignees.** (superseded by R34) **Change:** Assign takes numAssignees.
- **R24. Fallback handler.** (kept for escalation only, see R36) **Change:** the last participant in a chain became a FallbackHandler that assigns and completes. **Why:** a chain should always end with a decision.
- **R25. Include guards. Original:** a TIVIDY_ prefix. **Change:** the upper-case file name plus _H. **Why:** team convention.
- **R26. Strategies run in the order given. Original:** RuleAssignment always ran last. **Change:** strict order, and asking for too many assignees throws an exception (InsufficientParticipantsException since R41). **Why:** the special case added complexity. **Diagrams:** class diagram, activity diagram 2.
- **R27. Chains are set by Assign.** (participant source changed in R35) **Original:** the first Escalate or Approve built the chain. **Change:** Assign installs the chains and the other commands only move along them. **Why:** the chain is known at assignment. **Diagrams:** class and activity diagrams.
- **R28. Undo of Assign covers the chains. Change:** mementos record the chains, and undoing an Assign removes the chains it created. **Why:** Assign creates the chains, so undo must remove them.
- **R29. Pattern roles aligned with the taught structure.** (the Aggregate part was reversed in R31) **Change:** WorkflowPrototype, Builder and Aggregate were added, WorkflowMediator keeps its colleagues, and WorkMemento accessors became private with the originators as friends. **Why:** follow the taught structure and keep the caretaker's view of the memento narrow. **Diagrams:** class diagram.
- **R30. Director separated from the Facade. Problem:** the Facade was both Director and builder holder. **Original:** TividyFacade directed the builder with constructSimpleWorkflow and constructComplexWorkflow (R14). **Change:** a

WorkflowDirector holds the two recipes. The Facade forwards to it (constructIncidentResponseWorkflow, constructJobApplicationWorkflow) and exposes the builder for custom workflows. **Why:** each class has one job, and the GUI can build any workflow without a Director. **Diagrams:** class diagram, sequence diagram 2.

- **R31. Aggregate removed. Original:** an Aggregate interface that WorkflowDefinition also inherited (R29). **Change:** removed, and WorkComponent and WorkflowDefinition declare createIterator themselves. **Why:** a one-method base class added nothing, since WorkComponent is already the common interface. **Diagrams:** class diagram.
- **R32. Participants are shared, not owned. Original:** a clone copied every participant. **Change:** the application owns participants, workflows keep pointers, and a clone shares them. **Why:** one person can work in several workflows, so workload has to be counted across all of them. **Diagrams:** class and object diagrams.
- **R33. Strategies are passed per assignment. Original:** each WorkItem stored its strategies and the Builder had withStrategy (R10). **Change:** the Assign command passes the strategies to assignWork and nothing is stored, so withStrategy and addStrategy were removed. filter no longer takes the number of assignees. Selection happens after filtering through a select hook, LeastLoadedAssignment sorts by workload instead of dropping people, and RuleAssignment picks randomly in select. **Why:** the right rule can change for each assignment, and the item has nothing to remember. **Diagrams:** class diagram, activity diagram 2.
- **R34. Candidates come from the whole pool. Original:** the user passed a list of participants and a number (R23). **Change:** strategy assignment always uses the workflow's whole participant pool, and a separate participant list is passed only when no strategies are given. AssignCommand has a constructor for each case. **Why:** it is clear which people the strategies act on. **Diagrams:** class diagram, activity diagram 2.
- **R35. Chains are separate from assignees. Original:** chains were built from the participants left after the strategies, and handlers made their participant an assignee (R18, R24, R27). **Change:** the user supplies an escalation list and an approval list, strategies never filter them, handlers do not change assignees, and the decorator only records the current contact. The facade builds the LevelHandlers and adds the FallbackHandler to escalation chains. Handler gained copy() so clones and mementos can copy chains. **Why:** the people who escalate or approve are usually not the people who do the work, and they should not receive its workload. **Diagrams:** class diagram, activity diagrams 2 and 3.
- **R36. Escalation and approval rules. Original:** Approve moved along the chain, the fallback completed the item and approval had a fallback too. **Change:** the first Escalate or Approve moves the item to Escalated or Approval and gives the request to the first handler in the same step. Each later command moves one handler on. When the handler reached is the last one (the FallbackHandler for escalation, the last approver for approval) the item completes immediately. A Complete command stops an escalation, approval has no fallback, and an empty approval chain is refused. **Why:**

it matches how people hand work on, so N approvers need N Approve commands. **Diagrams:** state diagram, activity diagram 3.

- **R37. Escalate, then Approve. Original:** on an item with both decorators the escalation was completed with a separate Complete step before approval. **Change:** when an escalation ends, EscalatedWork calls approve and the item goes straight from Escalated to Approval. The builder fixes the wrapping order (Audited, Escalated, Approved, WorkItem), and an Approve during an active escalation is refused. **Why:** it removes a manual step and lets the escalation layer reach the approval layer. **Diagrams:** state diagram, activity diagram 3, class diagram.
- **R38. Restartable Rejected, no reassignment. Original:** Rejected was always final, and Escalated could go back to Assigned with assign(p). **Change:** a rejection from Approval can be restarted with Start, which returns the item to InProgress and restarts the chains. Rejections from other states stay final. Reassignment from Escalated was removed and Escalated gained approve. **Why:** approvers can reject for reasons that can be fixed, and reassignment clashed with moving along the chain. **Diagrams:** state diagram, activity diagram 3.
- **R39. Multi-level undo and layered mementos. Original:** undo of the last command (R12). **Change:** CommandHistory is a stack. A successful undo removes the command so the next one can be undone, and a refused undo leaves it on the stack. A command is undoable only if every affected component is still in the state the command left it. Complete, Escalate and Approve also snapshot the next step's items. A failed command rolls back only if a state actually changed. Mementos store one entry per decorator layer, and commands accept work items only. **Why:** undo must never leave later work half started. **Diagrams:** communication diagram, class diagram.
- **R40. The mediator keeps the order. Original:** components were linked through next, and the mediator also knew the order. **Change:** next was removed from WorkComponent, and WorkflowMediator keeps the ordered top-level steps and finds the next one itself. **Why:** the same information was stored twice, and decorators should not change a step's identity. **Diagrams:** class diagram, sequence diagram 1.
- **R41. Custom exceptions. Problem:** errors were only noted as “to be implemented”, and the GUI needs readable messages. **Change:** TividyException is the base class with eleven subclasses: InvalidTransition, UnsupportedOperation, InvalidAssignmentArgument, InsufficientParticipants, InvalidHandlerChain, ApprovalIncomplete, DuplicateId, ComponentNotFound, InvalidWorkflow, NotPristine and AuditEntryNotFound. **Why:** the GUI catches one base type and shows the message. **Diagrams:** class diagram.
- **R42. Visitors see decorators. Original:** decorators forwarded accept, so a visitor never saw the decoration. **Change:** WorkVisitor has a visit overload for each decorator, each decorator announces itself and then forwards, and TreePrinter was added beside ProgressReport. **Why:** the demo must print decorated items differently. **Diagrams:** class diagram.

- R43. Editable audit log. Original: AuditedWork only logged. Change: users can add, edit and delete entries, and an undo adds an “undone” entry. Why: people need to add notes and fix wrong ones. Diagrams: class diagram.
- R44. Ids and names come from the user. Problem: should the system make the ids? Original: an id generator was considered. Change: the user gives both, and the builder rejects repeats. Why: ids are a template type, and a generic generator could not make ints and strings alike.
- R45. Two concrete scenarios. Original: one general incident scenario with parallel groups (R22). Change: Incident Response (Power Outage) is five sequential steps with decorators on Investigation and Repair, and Job Application (Tech Company) has two parallel groups and an approved final step. Why: together they show every pattern in the demo: decorators and chains in the first, composite and parallel work in the second. Diagrams: object diagrams, activity diagram 3.
- R46. Workload and console output. Problem: LeastLoadedAssignment needs correct workloads, and the demo needs visible feedback. Change: a WorkItem raises its assignees' workload when assigned, releases it in an end state, and raises it again on a restart or an undo. Commands, state changes and chain steps print messages. Why: accurate counts and a readable demo.
- R47. Participant roles in the constructor. Problem: roles were added one at a time after the participant was created, which was awkward for the GUI. Original: Participant(id, name) plus an addRole function. Change: the constructor now takes the roles as a vector of strings, and addRole was removed. Why: a participant is complete as soon as it exists, and the GUI can pass everything in one call from a form. Diagrams: class diagram.

TESTING, DOCUMENTATION AND GITHUB WORKFLOW

GitHub repository

https://github.com/HaythamP/TIVIDY.git

Branching strategy

We kept a main branch and a dev branch, and every member also had a branch of their own. We worked independently on our own branches and merged them into dev through Pull Requests. Once everything in dev was working, we merged dev into main.

Testing

We used the GoogleTest library to check that our code works as intended. It compares outputs with expected values automatically, so we did not have to inspect print statements by hand. We wrote unit tests for each pattern, integration tests for combinations of patterns and two end-to-end tests (one per concrete scenario), all in a separate folder. The CI/CD pipeline relies on these tests to make sure only the right code gets merged, so broken code cannot reach the main branch.

Makefile

We wrote a Makefile that holds the commands our team uses most often, for running the tests and for checking for memory leaks. It makes sure we compile with the C++11 standard, builds the GoogleTest files with one command, and is what the CI/CD pipeline calls.

Documentation

Every pattern owner wrote Doxygen comments for their classes. Each header explains what the class does, who owns what and which pattern it plays.

TEAM CONTRIBUTION STATEMENT

Everyone wrote the header and source files of their own patterns, including the Doxygen comments and unit tests. Template Method sits inside WorkCommand and Handler, so it was done with Command and Chain of Responsibility.

Team Member	Contribution
Rayhaan	– Command, Memento – GUI – Activity Diagram 3
Tashil	– Strategy, Chain of Responsibility, Façade, main – Class Diagram – Report – Demo Slides, Demo Script
Kyle	– State, Iterator, Visitor – Activity Diagram 1, State Diagram, Object Diagrams, Communication Diagram – Docker
Haytham	– Builder, Mediator, exception classes, README – Integration tests and end-to-end tests – Activity Diagram 2, Sequence Diagram 2 – CI/CD pipeline
Oratile	– Composite, Decorator, Prototype – Sequence Diagram 1 – GUI

BUILD, RUN AND TEST INSTRUCTIONS

System Requirements & Dependencies

The dependencies required to build and verify TIVIDY mirror the packages installed in the project Dockerfile:

- **Operating System**: Linux (Ubuntu 20.04 recommended), macOS or Windows via WSL2 / Docker.
- **C++ Compiler**: g++ with support for -std=c++11, -Wall and -Werror.
- **Build System**: GNU Make 4.0+ (make), CMake 3.10+.
- **Testing Framework**: GoogleTest release-1.12.1 built with C++11 constraints. (The latest version of GoogleTest requires C++14 or later so we use an older verson of GoogleTest)
- **Memory Checking**: valgrind
- **Documentation Engine**: doxygen
- **System Utilities**: git, ca-certificates, pthread, curl and libasio-dev

Instructions To Run Inside Docker

```
# Clone the repository onto your machine
git clone https://github.com/HaythamP/TIVIDY.git
cd TIVIDY

# Ensure you are in the main branch
git checkout main

# Build the Docker Image
docker build -t tividy:latest .

# Launch the GUI
docker run --rm -p 8080:8080 --name tividy-gui tividy:latest

Open  http://localhost:8080 in a browser
Press Ctrl + C in the terminal to stop the server

# Commands for the Test Suites and Main:

# Run Unit Tests
docker run --rm tividy:latest make test-unit

# Run Integration Tests
docker run --rm tividy:latest make test-integration

# Run End to End Tests
docker run --rm tividy:latest make test-e2e
```

```
# Run Valgrind
docker run --rm tividy:latest make valgrind

# Run main.cpp
docker run --rm tividy:latest make run

# (Optional) rmi tividy:latest
docker rmi tividy:latest
```

Alternatively inside any terminal

```
# Docker must be installed and running

# Windows and Linux Users
docker run --rm -p 8080:8080 ghcr.io/haythamp/tividy:latest
```

Or

```
# Mac users
docker run --platform linux/amd64 --rm -p 8080:8080 ghcr.io/haythamp/tividy:latest
```

Open http://localhost:8080 in a browser

Press Ctrl + c in the terminal to stop the server

```
# Any commands from before still apply
```

BONUS MARKS

Docker

The Dockerfile installs the dependencies and runs the application inside a container. It is based on Ubuntu 22.04 and installs g++, make, cmake, git, ca-certificates, curl, Crow, libasio-dev, valgrind and gdb. The latest GoogleTest needs C++14 or later, which does not fit our C++11 build, so we installed an older GoogleTest version in the container. The container exposes port 8080 so that the GUI works, and the Dockerfile copies in the src, testing, public and samples folders and the Makefile.
We also published an image of our final build to the GitHub Container Registry. Anyone can run it, whatever their operating system, with a single docker run command (see Build, Run and Test Instructions). This is the simplest way to get a running copy of the system on any device that has the Docker engine.

CI/CD pipeline

Our test.yml file defines a job called build-and-test that runs when a pull request is made into dev. It builds a Docker image from the Dockerfile and then runs make test and make valgrind inside the container. A repository ruleset requires the build-and-test check, which GitHub Actions runs, to pass before a branch can be merged into dev. A second YAML file runs the unit, integration and end-to-end tests when code is merged into main.

REFERENCES

[1] IBM, “What is a workflow?” [Online]. Available: https://www.ibm.com/think/topics/workflow. [Accessed: Oct. 5, 2026].

[2] D. Hollingsworth, “The Workflow Reference Model,” Workflow Management Coalition, Document TC00-1003, Issue 1.1, 1995. [Online]. Available: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/refmodel/rmv1-16.html. [Accessed: Oct. 5, 2026].

[3] Camunda, “Versioning process definitions,” Camunda 8 Docs, Best Practices. [Online]. Available: https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/. [Accessed: Oct. 5, 2026].

[4] IBM, “Workflow definitions and workflow instances,” IBM OpenPages Version 9.0.0 documentation. [Online]. Available: https://www.ibm.com/docs/en/openpages/9.0.0?topic=fundamentals-workflow-definitions-workflow-instances. [Accessed: Oct. 5, 2026].

[5] LaunchNotes, “Workflow Diagram in Product Management and Operations.” [Online]. Available: https://www.launchnotes.com/glossary/workflow-diagram-in-product-management-and-operations. [Accessed: Oct. 5, 2026].

[6] IBM, “Routing concepts,” IBM Content Cortex Version 5.6.0 documentation. [Online]. Available: https://www.ibm.com/docs/en/content-cortex/5.6.0?topic=workflows-routing-concepts. [Accessed: Oct. 5, 2026].

[7] IBM, “Approval workflows overview,” IBM Apptio Planning documentation. [Online]. Available: https://www.ibm.com/docs/en/apptio-commercial/planning-standard/saas?topic=collaboration-approval-workflows-overview. [Accessed: Oct. 5, 2026].

[8] M. Ali, “How to define roles and responsibilities in a workflow,” Workhint Blog. [Online]. Available: https://blog.workhint.com/blog/how-to-define-roles-and-responsibilities-in-a-workflow/. [Accessed: Oct. 5, 2026].

[9] Unstructured, “Managing dependencies in data workflows: a practical guide,” Unstructured Insights. [Online]. Available: https://unstructured.io/insights/managing-dependencies-in-data-workflows-a-practical-guide. [Accessed: Oct. 5, 2026].

[10] D. M. R. Ferreira and J. J. Pinto Ferreira, “Developing a reusable workflow engine,” Journal of Systems Architecture, vol. 50, no. 6, pp. 309-324, 2004. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S1383762103001486.

[Accessed: Oct. 20, 2026].

[11] Docsie, “Audit Workflow.” [Online]. Available: https://www.docsie.io/blog/glossary/audit-workflow/. [Accessed: Oct. 22, 2026].

[12] MyBroadband, “Explosion causes widespread power outages in South African capital city,” 2026. [Online]. Available: https://mybroadband.co.za/news/energy/669718-explosion-causes-widespread-power-outages-in-south-african-capital-city.html. [Accessed: Oct. 5, 2026].

[13] African News Agency, “Tshwane power crisis continues after explosion at Watloo substation,” IOL, Sep. 30, 2026. [Online]. Available: https://iol.co.za/news/south-africa/2026-09-30-tshwane-power-crisis-continues-after-explosion-at-watloo-substation/. [Accessed: Oct. 5, 2026].

[14] MyBroadband, “Electricity rationing hits South Africa's capital,” 2026. [Online]. Available: https://mybroadband.co.za/news/energy/670006-electricity-rationing-hits-south-africas-capital.html. [Accessed: Oct. 5, 2026].

[15] TimesLIVE, “Tshwane works to restore power after Waltloo-Njala interruption,” Oct. 1, 2026. [Online]. Available: https://www.timeslive.co.za/news/south-africa/2026-10-01-tshwane-works-to-restore-power-after-waltloo-njala-interruption/. [Accessed: Oct. 5, 2026].

[16] Tyto Insights, “Tyto Insights - Pretoria-based data consultancy for South Africa.” [Online]. Available: https://tyto.africa. [Accessed: Oct. 5, 2026].